Jenkins

Jenkins is an open-source automation tool used to automate the build, testing, and deployment of applications. It is mainly used to implement Continuous Integration (CI) and Continuous Delivery/Deployment (CI/CD) pipelines.

It reduces manual effort and ensures faster, more reliable software delivery.

Key Features

- Open-source and free
- Automates Build, Test, and Deployment
- Supports Continuous Integration (CI)
- Supports Continuous Delivery/Deployment (CD)
- Integrates with Git, Maven, Docker, Selenium, JUnit, etc.
- Supports thousands of plugins
- Can schedule jobs or trigger them automatically

Why do we need Jenkins?

Jenkins is needed to automate the software development process. It reduces manual work by automatically building, testing, and deploying applications whenever code changes are made, ensuring faster and more reliable software delivery.

Where do we use Jenkins?

Jenkins is used in DevOps and CI/CD pipelines to:

- Build applications
- Run automated tests
- Generate reports
- Deploy applications
- Automate repetitive tasks

We use Jenkins in software development projects to automate the build, testing, and deployment processes. It is commonly used in CI/CD pipelines to integrate code changes, execute automated tests, and deploy applications automatically**.**

Real-Time Example

Suppose a developer pushes code to GitHub.

Developer
↓
GitHub
↓
Jenkins
↓
Build
↓

Run Selenium/API Tests
↓
Deploy

Jenkins automatically performs all these steps without manual intervention

Jenkins Workflow

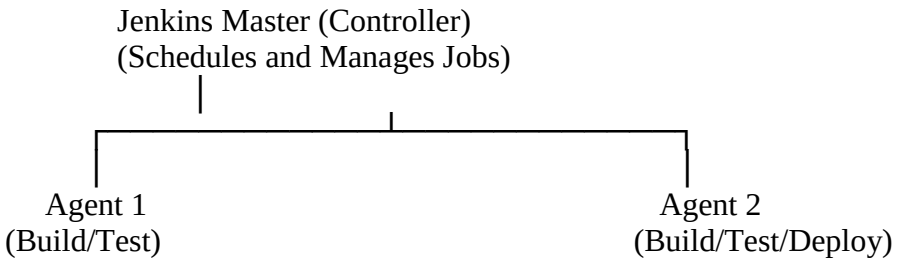
A developer pushes code to GitHub. Jenkins detects the changes using a Build Trigger, checks out the latest code, builds the application, runs automated tests, generates reports, and deploys the application if the build is successful.

Jenkins Architecture

Jenkins follows a **Controller-Agent architecture**(Master-slave)

Jenkins Architecture consists of a **Master (Controller)** and one or more **Agents (Slaves)**. The Controller manages Jenkins, schedules jobs, and assigns tasks to Agents. Agents execute the build, test, and deployment tasks and send the results back to the Controller.

Master-slave (or) controller-agent



1. Controller (Master)

The Controller is the central Jenkins server that manages Jenkins, schedules jobs, assigns tasks to Agents, stores configurations, and monitors the build process

Responsibilities of Master

- Manages Jenkins.
- Schedules build jobs.
- Assigns jobs to Agents.
- Stores job configurations.
- Displays build results.

2.Agent

An Agent executes the build, test, and deployment tasks assigned by the Controller and returns the results after execution.

Responsibilities of Agent

- Executes build jobs.
- Runs automation tests.
- Performs deployments.
- Sends build results back to the Master.

Why do we use Multiple Agents?

We use multiple Agents to:

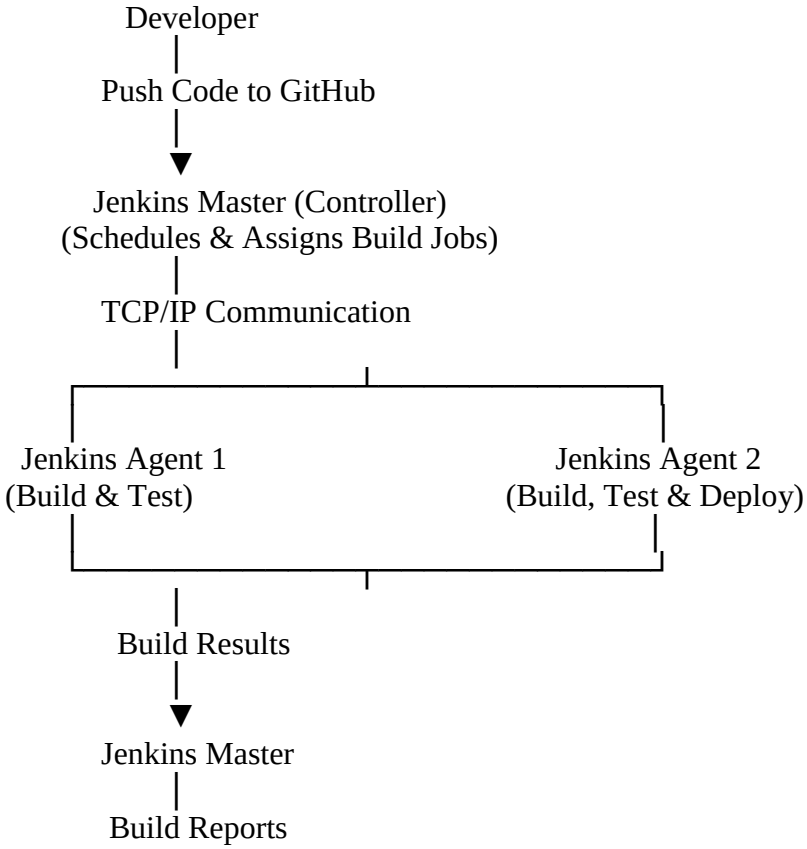
- Run multiple jobs in parallel.
- Distribute the workload.
- Improve performance.
- Reduce build time.
- Execute builds on different operating systems (Windows, Linux, macOS).

Communication Between Controller and Agent

The Controller and Agent communicate using the **TCP/IP protocol**.

- The Controller sends build jobs to the Agent.
- The Agent executes the jobs.
- The Agent sends the build results back to the Controller.

Jenkins Architecture Diagram



Why does Jenkins use Controller-Agent architecture instead of a single server?

Jenkins uses a Controller-Agent architecture to distribute the workload across multiple machines. If all build, test, and deployment tasks run on a single server, it can become overloaded, resulting in slower performance and longer build times. By using multiple Agents, Jenkins can execute multiple jobs in parallel, improve performance, support different operating systems, and provide better scalability.

What is a Job?

A Job is a task or a set of instructions that Jenkins executes automatically, such as building an application, running tests, generating reports, or deploying an application.

Types of Jenkins Jobs

- **Freestyle Project** – Simple jobs with GUI configuration.
- **Pipeline Job** – Uses a Jenkinsfile to define the workflow as code.
- **Multibranch Pipeline** – Automatically creates pipeline jobs for multiple G

What are Build Triggers?

Build Triggers are mechanisms that automatically start a Jenkins job when a specific event or condition occurs, such as a code change, a scheduled time, or the completion of another job.

Why do we need Build Triggers?

Without Build Triggers

- Developer pushes code.
- Someone manually clicks **Build Now**.
- This wastes time and delays testing and deployment.

With Build Triggers

- Developer pushes code.
- Jenkins automatically starts the build.
- Tests run immediately.
- Faster feedback and automation.

Types of Build Triggers

1. Build Now (Manual Trigger)

Build Now is used to manually start a Jenkins job whenever required.

2. Build Periodically

It automatically runs a Jenkins job at scheduled times using a **Cron expression**.

Example:

H 2 * * *

This runs the Jenkins job **every day at approximately 2:00 AM**.

3. Poll SCM

Poll SCM allows Jenkins to periodically check the source code repository for changes. If changes are detected, Jenkins automatically starts the build.

Example:

H/5 * * * *

This means Jenkins checks the Git repository **every 5 minutes**. If changes are detected, it triggers the build.

4. GitHub Webhook

A **GitHub Webhook** instantly notifies Jenkins whenever code is pushed to the GitHub repository. Jenkins immediately starts the build without waiting for periodic checks.

Interview Point:
GitHub Webhook is **event-based** and faster than Poll SCM because GitHub immediately notifies Jenkins after a code push.

5. Build after Other Projects are Built

This trigger automatically starts a Jenkins job **after another Jenkins job completes successfully**.

Quick Revision

- **Build Trigger** → Decides **when** Jenkins should start a job.
- **Build Now** → Manual trigger.
- **Build Periodically** → Runs jobs at scheduled times using a Cron expression.
- **Poll SCM** → Jenkins checks the Git repository periodically for changes.
- **GitHub Webhook** → GitHub instantly notifies Jenkins after a code push.
- **Build after Other Projects are Built** → Starts a job after another job completes successfully.

Which is better: Poll SCM or GitHub Webhook?

GitHub Webhook is better because it triggers the build immediately after a code push. Poll SCM checks the repository at regular intervals, so there may be a delay before the build starts.

What are Jenkins Plugins?

Jenkins Plugins are extensions that add additional features and allow Jenkins to integrate with external tools such as Git, Maven, Docker, Selenium, and JUnit.

Why are Plugins required?

Plugins extend Jenkins' functionality. They enable Jenkins to integrate with different tools, automate tasks, publish reports, and support CI/CD workflows.

Name some commonly used Jenkins Plugins.

- **Git Plugin** → Integrates with Git.
- **Maven Plugin** → Builds Maven projects.
- **Pipeline Plugin** → Supports Jenkins Pipelines.
- **Docker Plugin** → Integrates with Docker.
- **JUnit Plugin** → Publishes test reports.
- **HTML Publisher Plugin** → Publishes HTML reports.

- **Email Extension Plugin** → Sends email notifications.

What is a Jenkins Pipeline?

A Jenkins Pipeline is a sequence of automated stages that define the complete CI/CD workflow, including building, testing, and deploying an application.

Types of Jenkins Pipeline

1. Declarative Pipeline

A Declarative Pipeline is a structured pipeline written using predefined syntax. It is easy to read, easy to maintain, and is the most commonly used pipeline.

Features

- Simple syntax
- Easy to understand
- Less coding
- Recommended for most projects

2. Scripted Pipeline

A Scripted Pipeline is written using the Groovy programming language. It provides more flexibility and is mainly used for complex CI/CD workflows.

Features

- More flexible
- Supports complex logic

- Requires Groovy knowledge
- Used for advanced projects

What is a Stage?

A Stage is a major phase in a Jenkins Pipeline, such as Checkout, Build, Test, Package, or Deploy.

Each stage contains one or more steps.

Common Pipeline Stages

Stage	Purpose
Checkout	Download source code from Git
Build	Compile the project
Test	Execute automated tests
Package	Create JAR/WAR or Docker image
Deploy	Deploy the application

What is an Agent?

The **Agent** specifies **where** the pipeline will run.

Example:

agent any

This means Jenkins can run the pipeline on any available agent.

What is Steps

A **Step** is an individual task inside a stage.

Examples

- Run Maven command
- Execute Selenium tests
- Print a message

What is Post

The **Post** section defines actions that execute after the completion of pipeline.

Examples

- Send email notification
- Archive reports
- Clean workspace

Why are Pipelines better than Freestyle Jobs?

Freestyle Job	**Pipeline**
Configured through the UI	Defined as code
Difficult to track changes	Easy to version control using Git
Limited flexibility	Highly flexible
Harder to reuse	Reusable and maintainable

Prefer pipeline over freestyle job

What is a Jenkinsfile?

A **Jenkinsfile** is a text file that contains the Jenkins Pipeline code. It defines the entire CI/CD workflow, including stages such as build, test, and deployment. It is usually stored in the project's Git repository.

Why do we use a Jenkinsfile?

- Automates the **CI/CD pipeline**.
- Stores the **pipeline code in Git** for version control.
- **easy to modify and reuse**.
- Ensures everyone follows the **same build process**.

Where is the Jenkinsfile stored?

The Jenkinsfile is usually stored in the root directory of the project's Git repository.

Why Jenkinsfile instead of configuring Pipeline in Jenkins UI?"

Configuring the pipeline in the Jenkins UI is difficult to track and maintain. A Jenkinsfile stores the pipeline as code in Git, making it version-controlled, reusable, easy to modify, and easy to share with the team.

Real-Time Maven Jenkinsfile(Declaritive pipeline)

For a Java Maven project:

```
pipeline {
    agent any
```

```
    stages {

        stage('Checkout') {
            steps {
                git 'https://github.com/example/project.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean compile'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Package') {
            steps {
                sh 'mvn package'
            }
        }
    }
}
```

1.pipeline

The pipeline block is the starting point of every Declarative Pipeline.

It contains the complete CI/CD workflow.

2. agent

agent any

This tells Jenkins to execute the pipeline on **any available agent**.

3. stages

The stages block groups all the major phases of the pipeline.

Examples:

- Build
- Test
- Deploy

4. stage

Each stage represents one major step.

Example:

stage('Build')

This stage is responsible for building the application.

5. steps

The steps block contains the actual commands to execute.

Example:

```
steps {
   echo 'Building the project...'
}
```

What happens?

- **Checkout** → Downloads the latest code from GitHub.
- **Build** → Compiles the project.
- **Test** → Runs automated tests.
- **Package** → Creates the application package (JAR/WAR).

Interview Summary

- **Jenkinsfile** → Contains the Pipeline code.
- **Stored in Git** → Along with the project source code.
- **pipeline** → Starting block.
- **agent** → Specifies where the pipeline runs.
- **stages** → Groups the major phases.
- **stage** → Represents one phase (Build, Test, etc.).
- **steps** → Contains the commands to execute.

What is Cron Expression?

Answer:

A Cron Expression is a scheduling format used in Jenkins to execute jobs automatically at specific dates and times.

What happens when a build fails?

Very common interview question.

Answer:

If a build fails, Jenkins marks the build as failed, stops the pipeline (unless configured otherwise), generates reports, and sends notifications to the team. Developers analyze the logs, fix the issue, commit the changes, and trigger the build again.

Build Status

Know the common statuses:

- Success ✅
- Failure ❌
- Unstable ⚠️
- Aborted ⛔

Jenkins Advantages

Answer:

- Open source.
- Easy to use.
- Supports CI/CD.
- Large plugin ecosystem.
- Supports distributed builds.
- Cross-platform.
- Easy integration with DevOps tools.

Continuous Integration (CI)

Continuous Integration (CI) is a software development practice in which developers frequently merge their code into a shared repository. Whenever code is committed, it is automatically built and tested to detect issues early.

Simple Interview Answer

Continuous Integration (CI) is the process of automatically building and testing code whenever developers push changes to a shared repository like GitHub.

Why do we need CI?

Before CI:

- Developers worked on code separately.
- Code was merged after several days or weeks.
- Integration issues were discovered late.
- Fixing bugs took more time.

With CI:

- Developers merge code frequently.
- Detects bugs early
- Reduces integration issues
- Automates build and testing after every commit
- Software quality improves.
- Provides quick feedback and saves time

flow

Developer writes code

|

Git Commit

|

Git Push

|

GitHub Repository

|

Jenkins detects the change

|

Downloads latest code

|

Builds the project

|

Runs automated tests

|

Build Success / Failure Report

How Jenkins Implements CI

Jenkins is one of the most popular tools used to implement CI.

Workflow:

1. Developer pushes code to GitHub.
2. GitHub notifies Jenkins (Webhook) or Jenkins checks using Poll SCM.
3. Jenkins downloads the latest code.
4. Builds the project (using Maven/Gradle).

5. Runs automated tests (JUnit/Selenium/TestNG).
6. Publishes test reports.
7. Sends notifications if the build fails or succeeds.

Continuous Delivery (CD)

Continuous Delivery (CD) is a software development practice in which the application is automatically built and tested, and kept ready for deployment. However, deployment to the production environment requires manual approval.

Simple Interview Answer

Continuous Delivery (CD) automatically builds and tests the application and keeps it ready for deployment. However, deployment to production requires manual approval.

Continuous Deployment (CD)

Continuous Deployment (CD) is a software development practice in which the application is automatically built, tested, and deployed to the production environment without manual approval.

Simple Interview Answer

Continuous Deployment (CD) automatically builds, tests, and deploys the application to the production environment without any manual approval.

Easy Trick to Remember

- **Continuous Integration (CI) → Build + Test**
- **Continuous Delivery (CD) → Build + Test + Ready for Deployment (Manual Approval)**
- **Continuous Deployment (CD) → Build + Test + Automatically Deploy to Production (No Manual Approval)**

Continuous Delivery vs Continuous Deployment

Continuous Delivery	Continuous Deployment
Manual approval is required before production deployment.	No manual approval is required.
Production deployment is a business decision.	Production deployment is fully automated.
Lower risk.	Faster releases but requires high confidence in automated testing.
Common in banking, healthcare, and government projects.	Common in startups and cloud-native applications.